SENO EN PUNTO FIJO PARA LA LC-3 (LAB 3b)

“Chebyshev–Padé”

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIA:

IC221 - Arquitectura de computadoras

DOCENTES:

Dr. Ing. Roberto E. Carballo

Dr. Ing. Aldo J. Benitez.

Ing. Roberto N. Schuster

Ing. Patricia V. Feltan

Ing. Gustavo O. Rossler

Ing. Eduardo Díaz

Sr. Joaquín Nahuel Giménez

Sr. Joaquín Zuk

ALUMNOS:

Meretz Mariana Belén

Meretz Eliana Marife

GRUPO: 30

ÍNDICE

Contenido

PROBLEMA N°1 3
1.1. DESARROLLO 3
1.1.1. Introducción 3
1.1.2. Resultados 14
1.1.3. Discusión 19
2. CONCLUSIONES 20
3. REFERENCIAS BIBLIOGRÁFICAS 20

PROBLEMA N°1

1.1. DESARROLLO

1.1.1. Introducción

El programa recibe un ángulo y devuelve su seno, lo difícil es que la LC-3, es muy simple: Es de 16 bits, no sabe multiplicar, no sabe dividir y no entiende números con coma. Solo sabe sumar, restar y mover números de lugar. Por eso el programa tiene que armar todo con esas pocas operaciones: calcula el seno usando solo sumas, restas y desplazamientos de bits, sin decimales y sin usar el seno que ya trae un lenguaje de programación. Para guardar números con coma se recurre a la aritmética de punto fijo.

El ángulo de entrada se recibe en formato Q3.12 (1 bit de signo, 3 bits enteros y 12 decimales), que cubre de −8 a casi 8 radianes con un paso de 0,000244 rad (1/4096) y permite expresar ángulos de más de una vuelta completa (2π ≈ 6,28). La salida está en formato Q1.14 (1 bit de signo, 1 bit entero y 14 decimales). En todo el informe los formatos Q se escriben sin contar el bit de signo, que siempre está presente.

El resultado es una aproximación, no el seno exacto, pero se equivoca muy poco: como máximo en unas 4 unidades de las 16384 (3,75 en las mediciones), es decir, alrededor del 0,03 %.

Origen de la fórmula: el método híbrido de Chebyshev–Padé

La fórmula con la que se calcula el seno:

$$\text{sen}(x) \approx x \cdot \frac{(1 - A \cdot u)}{(1 + B \cdot u)} \qquad \text{con } u = \frac{x^2}{2}$$

Esta no se eligió al azar: proviene del método híbrido de Chebyshev–Padé. Es un cociente entre dos expresiones simples en x (una fracción racional): arriba x · (1 − A·u), un polinomio de grado 3 en x, y abajo 1 + B·u, un polinomio de grado 2. El nombre “híbrido” viene de que combina dos ideas: la forma de una aproximación de Padé y el criterio de error de Chebyshev.

La parte de Padé aporta la forma. Una aproximación de Padé arma una fracción de polinomios cuyo desarrollo en serie coincide con el de la función cerca de x = 0, igual que un polinomio de Taylor pero usando una fracción. Para el seno, la fracción de grado 3 sobre 2 es sen(x) ≈ x · (1 − 7x²/60) / (1 + x²/20), que equivale a A = 7/30 y B = 1/10 en la fórmula del programa. *Es muy precisa cerca de 0, pero su error crece a medida que x se aleja del origen y es máximo justo en el borde, en π/2*.

La parte de Chebyshev aporta el ajuste. El criterio de Chebyshev (aproximación minimax) no pide que el error sea nulo en un punto, sino que el error más grande de todo el intervalo [0, π/2] sea el menor posible. La mejor aproximación bajo ese criterio tiene una firma reconocible: el error toca su valor máximo varias veces con signos alternados (+, −, +, −) y con alturas iguales (teorema de equioscilación de Chebyshev). Por eso, en lugar de usar los coeficientes exactos de Padé, se ajustaron A y B hasta repartir el error de forma pareja en todo el intervalo; así salieron A = 0,2241 y B = 0,1107. Estos coeficientes se calculan con un procedimiento iterativo (algoritmo de Remez), y en este trabajo se buscaron directamente sobre números enteros de 14 decimales.

Cuánto se gana. Medido con aritmética real, antes de cuantizar, en el intervalo [0, π/2]: con los coeficientes exactos de Padé el error máximo es de 0,0042 (unas 69 unidades de 1/16384), con un polinomio de Taylor de grado 5 es de 0,0045 (74 unidades) y con los coeficientes de Chebyshev usados aquí baja a 0,00014 (2,3 unidades), unas 30 veces menos. El error además ya no crece hacia el borde: aparecen picos de signo alterno cerca de x = 0,83 y x = 1,39 y en el extremo π/2, de alturas parecidas (entre el 73 % y el 100 % del máximo; no son exactamente iguales porque los coeficientes están redondeados a enteros). Los 3,75 de error máximo medidos en los resultados incluyen, además, los redondeos de las multiplicaciones y de la división en enteros.

Por qué es una buena fórmula. Tiene un error pequeño y parejo; usa solo cuatro multiplicaciones y una división, sin tablas grandes de valores ni un bucle por cada dígito del resultado; y, como suele ocurrir con las fracciones racionales, alcanza precisión similar a la de un polinomio con menos términos. Eso la hace adecuada para una computadora tan simple como la LC-3.

Cómo guardar números con coma sin comas

La computadora solo guarda números enteros, así que se usó un truco: multiplicar cada valor por una potencia de 2 y guardar el entero resultante. El ángulo de entrada se multiplicó por 4096 (formato Q3.12) y la salida, junto con los cálculos internos, por 16384 (formato Q1.14). Por ejemplo, 1,0 se guarda como 4096 en la entrada y como 16384 en la salida; 0,5 se guarda como 2048 y 8192; y π/2 (≈ 1,5708) se guarda como 6434 y 25736.

Es como cuando una tienda guarda los precios en centavos: $2,50 se anota como 250 y así nadie necesita la coma. Aquí la “moneda pequeña” vale 1/4096 en la entrada y 1/16384 en la salida. Para volver al número real se divide entre 4096 o entre 16384, según el caso. Q3.12 representa números de −8 a casi 8 (1 bit de signo, 3 enteros y 12 decimales) y Q1.14 representa de −2 a casi 2 (1 bit de signo, 1 entero y 14 decimales).

Se usaron formatos distintos porque el seno solo toma valores entre −1 y 1, de modo que la salida puede dedicar más bits a los decimales, mientras que el ángulo necesita más bits enteros para llegar a ±8 radianes (más de una vuelta completa, 2π ≈ 6,28). La Tabla 1 muestra algunos valores.

Tabla 1: Ejemplos de valores en cada formato

Valor real	Entrada Q3.12 (× 4096)	Salida Q1.14 (× 16384)
0,5	2048	8192
1,0	4096	16384
π/4 ≈ 0,785	3217	12868
π/2 ≈ 1,571	6434	25736
π ≈ 3,142	12868	no cabe (máximo 1,99994)

Reducción del ángulo

Calcular el seno de cualquier ángulo es difícil, pero entre 0 y π/2 (0° a 90°) existe una fórmula muy buena. Por eso se llevó primero cualquier ángulo a esa zona con cuatro reglas del seno, todas aplicadas en Q3.12:

- Un ángulo negativo tiene el mismo seno que el positivo con el signo cambiado, porque sen(−x) = −sen(x). Se trabajó con el valor absoluto |x| y se guardó el signo para invertirlo al final.

- Si |x| > 2π (360°), sen(x) = sen(x − 2π). Por ejemplo, sen(7 rad) = sen(0,717 rad): se resta una vuelta completa. Como |x| ≤ 8, basta restarla una sola vez.
- Si x > π (180°), sen(x) = −sen(x − π). Por ejemplo, sen(200°) = −sen(20°): se restó π y se invirtió el signo guardado.
- Si x > π/2 (90°), sen(x) = sen(π − x). Por ejemplo, el seno de 120° es igual al de 60°: todos los ángulos entre 90° y 180° se “reflejan” hacia la zona buena.

Es como doblar una hoja por la mitad: basta con saber calcular un pedazo de la curva y lo demás es el reflejo. Como |x| ≤ 8 < 4π, restar 2π una sola vez alcanza. Las constantes se expresaron en Q3.12: 2π = 25736, π = 12868 y π/2 = 6434. Al terminar, el ángulo se pasó de Q3.12 a Q1.14 con dos desplazamientos a la izquierda (x << 2), que es exacto porque x ≤ π/2 cabe en Q1.14.

```
uint16_t ax = (x < 0) ? (uint16_t)(0u - (uint16_t)x) : (uint16_t)x;
int neg = (x < 0);
if (ax > TWO_PI_Q12) ax -= TWO_PI_Q12;    /* sen(x) = sen(x - 2pi) */
if (ax > PI_Q12) { ax -= PI_Q12; neg ^= 1; } /* -sen(x - pi) */
if (ax > HALF_PI_Q12) ax = PI_Q12 - ax;   /* sen(x) = sen(pi - x) */

ax <<= 2;                                 /* Q3.12 -> Q1.14 */
```

El único caso especial es x = −8,0 (entero −32768): su valor absoluto, 32768, no cabe como entero con signo de 16 bits. En C se resolvió usando enteros sin signo. En ensamblador no hace falta un caso aparte: la resta |x| − 2π siempre da un resultado representable (para 32768 da 7032, es decir, 1,717 rad) y se obtiene el signo correcto.

Aproximación racional

Con el ángulo ya en la zona buena, se aproximó el seno con una fracción:

```
sen(x) ≈ x · (1 − A·u) / (1 + B·u)          con u = x²/2
```

A = 3672/16384 ≈ 0,2241 y B = 1814/16384 ≈ 0,1107 son dos números que se buscaron (ajuste minimax) para que la fracción se parezca lo más posible al seno verdadero. Esto es una aproximación, no el valor exacto. Calcularla requiere cuatro multiplicaciones y una división, y para eso se construyeron dos funciones: mulu y divf.

Función mulu: multiplicar dos números

Qué hace. Recibe dos números positivos, ya en Q1.14, y devuelve su producto en el mismo formato.

Por qué hace falta. La LC-3 no sabe multiplicar.

Cómo lo hace. Multiplica, solo con sumas, pero en binario. Por ejemplo, 13 × 5 es lo mismo que 13 + 13 + 13 + 13 + 13, pero el programa lo hace con menos pasos: escribe 5 en binario (101) y recorre sus bits de izquierda a derecha. En cada paso duplica lo acumulado y, si el bit vale 1, suma 13.

```
5 = 101   ->   13 (bit 1)   ->   26 (bit 0)   ->   2·26 + 13 = 65 (bit 1)
```

Ese es todo el bucle: en cada vuelta se duplica el acumulado y se revisa un bit del multiplicador; si vale 1, se suma el multiplicando. Así el multiplicando nunca se desplaza y ocupa una sola palabra, lo que deja libres registros (la LC-3 solo permite usar R0 a R5 como registros de trabajo).

El problema del tamaño. Multiplicar dos números de 15 bits puede dar 30 bits, y la LC-3 solo guarda 16. Es como querer escribir 1.000.000 en un cuaderno donde cada casilla solo aguanta 5 cifras. La solución es guardar el acumulador en dos cajas (hi y lo), como cuando se separan "miles" y "unidades" al escribir una cifra larga. Cada caja usa 15 bits y la pareja representa hi · 32768 + lo.

El "me llevo una". Cuando lo se pasa de su límite, ya sea al duplicar el acumulado o al sumar el multiplicando, el bit 15 se enciende. Es la señal de acarreo: se apaga ese bit y se suma 1 a hi.

El corte y el redondeo. Al multiplicar dos números en Q1.14, el resultado queda multiplicado de más por 16384, así que hay que quitar 14 bits (>> 14). Antes de cortar, al terminar el bucle, se suma media unidad, para redondear al valor más cercano y no siempre hacia abajo, como redondear 2,6 a 3 en vez de dejarlo en 2. Así el error es más pequeño.

Dos trucos de rapidez:

- Se pone el menor de los dos números como multiplicador (if (a < b) swap), para que el bucle tenga menos vueltas.
- El bucle empieza en el primer bit en 1 del multiplicador: los ceros iniciales se saltean con un ciclo corto. Así da tantas vueltas como bits tiene el menor de los dos números. Si ese número es 0, el producto es 0 y se sale de inmediato.

El parámetro half. Es un interruptor entre dos cortes: 14 bits (producto normal) o 15 bits (producto dividido entre 2). El de 15 sirve para calcular $x^2/2$ de una sola vez.

Función divf: dividir dos números

Qué hace. Recibe n y d (con n ≤ d) y devuelve n/d, también en Q1.14.

Por qué hace falta. La LC-3 tampoco sabe dividir.

Cómo lo hace. Con la división larga de la escuela, pero en binario. En cada vuelta se pregunta "¿cabe el divisor en lo que me queda?". Si cabe, ese dígito del resultado es 1 y se resta; si no, es 0. En binario solo existen dos dígitos, así que cada pregunta se responde con un sí o un no.

```
duplico el resto -> ¿cabe d? -> sí: resto -= d, bit = 1
                                no: bit = 0
```

Cuántas vueltas. 14, una por cada bit después de la coma. Como el resultado es menor o igual que 1, no hay parte entera que calcular.

El redondeo. Al final, si lo que sobra es al menos la mitad del divisor, se suma 1 al resultado, de modo que queda en el valor más cercano.

Por qué es tan corta. Solo se llama cuando n ≤ d y ambos son positivos. Eso lo garantiza sen_q12, así que no hace falta revisar signos ni división entre cero.

Función sen_q12: la función principal

Qué hace. Recibe un ángulo en Q3.12 y devuelve su seno en Q1.14. Usa a mulu y divf como ayudantes y tiene tres etapas.

Etapa 1: preparar el ángulo. La fórmula solo es buena entre 0 y π/2, por lo que primero se aplican las cuatro reglas descritas en la reducción del ángulo: valor absoluto con el signo guardado, resta de 2π, resta de π y reflexión π − x. Un caso aparte es el ángulo −8,0, cuyo valor absoluto no cabe en 16 bits con signo y se trata como número sin signo. Al final de esta etapa, el ángulo se pasa de Q3.12 a Q1.14.

Etapa 2: evaluar la fórmula. Con el ángulo ya listo se calcula:

```
u   = x²/2              (mulu con half = 1)
num = 1 − A·u            (mulu)
den = 1 + B·u            (mulu)
sen = x · num/den        (divf, luego mulu)
```

Son cuatro multiplicaciones y una división. A y B son las dos constantes buscadas para que la fracción se parezca lo más posible a la curva del seno.

Etapa 3: arreglar el resultado. Si el resultado se pasa de 1,0 por el error de la aproximación, se recorta a 1,0. Después se aplica el signo que se había guardado.

Cómo encajan las tres funciones

```
sen_q12  --usa-->  mulu  (4 veces)
         --usa-->  divf  (1 vez)
```

Las tres funciones tienen una responsabilidad propia. sen_q12 se ocupa de la matemática del seno y de dejar los datos listos. mulu y divf son las dos "herramientas básicas" que reemplazan lo que la LC-3 no trae.

De C a ensamblador

El ensamblador es lo mismo traducido a las instrucciones que la LC-3 entiende, una por una. Los dos hacen exactamente las mismas operaciones y dan el mismo resultado, bit por bit, lo que se comprobó con las 65 536 entradas posibles, incluidos los extremos. La diferencia es que en ensamblador no existen los if ni los for: todo se hace con saltos. Además, solo se pueden usar los registros R0 a R5 como registros de trabajo (R6 funciona como pila y R7 guarda la dirección de retorno de JSR y RET), por lo que algunos datos se guardaron en memoria aparte y mulu se escribió multiplicando de izquierda a derecha para que el multiplicando ocupe un solo registro.

Convención de llamada y registros

El programa principal (main) carga el ángulo en R0 y llama a la rutina con JSR SEN_Q12; la rutina devuelve el seno también en R0. R0 es, por lo tanto, el único registro cuyo valor cambia.

Para no dañar al programa que la llama, SEN_Q12 guarda en memoria (con ST) los valores que tenían R1 a R7 al entrar y los restaura (con LD) justo antes del RET. R7 se guarda primero porque cada JSR interno (a mulu y divf) lo pisa con una nueva dirección de retorno, y se restaura al final para que el RET vuelva al lugar correcto.

R6 (pila) y R7 (retorno) no se usan nunca como registros de trabajo: R6 solo se guarda y se restaura sin modificarse, y R7 solo lo modifican las instrucciones JSR. Las rutinas internas mulu y divf trabajan únicamente con R0 a R5, no llaman a otras rutinas (por eso no necesitan guardar R7) y retornan con RET. En las 65 536 entradas se comprobó que, al terminar, R1 a R7 tienen los mismos valores que antes de la llamada.

El código en ensamblador, paso a paso

A continuación se recorre el código completo (archivo seno_q12in.asm) en el mismo orden en que se ejecuta. Cada bloque se muestra como imagen y luego se explica. Dos ideas ayudan a leerlo: la LC-3 no tiene resta ni multiplicación, así que para restar se suma el opuesto (NOT y luego +1, o una constante que ya está guardada en negativo); y los saltos (BRn, BRz, BRp y sus combinaciones) dependen del signo del último resultado calculado.

Paso 1: programa principal y entrada a la rutina. El programa principal carga el ángulo en R0 (LD), llama a la rutina con JSR SEN_Q12 y guarda en RESULTADO el seno que vuelve en R0. El ángulo de prueba, 3217, es π/4 en Q3.12 y el resultado es 11584 (el ideal es 11585). Al entrar, SEN_Q12 guarda R1 a R7 en memoria con siete instrucciones ST, para devolverle al programa principal sus valores intactos al final. R7 queda guardado antes del primer JSR interno, que lo va a pisar con una dirección de retorno nueva.

```
1          .ORIG x3000
2
3          LD   R0, ANGULO_TEST     ; el angulo siempre se carga en R0
4          JSR  SEN_Q12             ; el resultado vuelve en R0
5          ST   R0, RESULTADO       ; sen(pi/4) ~ 11584 (ideal 11585)
6          HALT
7
8  ANGULO_TEST .FILL #3217          ; pi/4 en Q3.12 (0.7854 * 4096)
9  RESULTADO   .BLKW #1
10
11 ; ================================================================
12 ; SEN_Q12 : R0 = sen(R0)      (Q3.12 -> Q1.14)
13 ; ================================================================
14 SEN_Q12
15         ST   R1, SN_R1           ; guardar los registros del llamador
16         ST   R2, SN_R2
17         ST   R3, SN_R3
18         ST   R4, SN_R4
19         ST   R5, SN_R5
20         ST   R6, SN_R6
21         ST   R7, SN_R7           ; (R7 se pisa en cada JSR interno)
```

Código 1: Programa principal y guardado de los registros del llamador

Paso 2: reducción del ángulo a [0, π/2]. Es la traducción de las reglas del seno que se vieron en la metodología:

- Signo (líneas 2 a 7). R4 es la bandera de signo: 0 es positivo y 1 es negativo. Si x es negativo, se obtiene |x| con NOT y +1 (complemento a dos) y se pone R4 en 1. Para x = −8,0 el resultado es x8000, que se interpreta como 32768 sin signo.
- Resta de 2π (líneas 8 a 11). Se suma la constante −25736 (SN_N2PI), guardada ya en negativo. Si |x| − 2π es menor o igual que cero (BRnz), el ángulo no cambia; si es positivo, esa diferencia pasa a ser el nuevo |x|. El resultado de esta suma siempre queda entre −25736 y 7032, así que su signo es confiable incluso para 32768.
- Resta de π (líneas 12 a 17). Igual, con −12868. Si se restó, se invierte la bandera de signo: NOT R4 y AND con 1 dejan solo el último bit, porque sen(x) = −sen(x − π).
- Reflexión (líneas 18 a 24). Se compara con π/2 (−6434). Si lo supera, se calcula π − |x| con NOT, +1 y la suma de π.
- Escalado y salida temprana (líneas 25 a 29). Dos ADD R0,R0,R0 desplazan el ángulo dos bits a la izquierda y lo pasan de Q3.12 a Q1.14. Si quedó en 0, el seno es 0 y se

salta a SN_EXIT. Si no, el signo y el ángulo se guardan en memoria (SN_SIGN y SN_X), porque R4 y R0 se van a reutilizar.

```
        ; ---- 1) Reduccion de rango (Q3.12) a [0, pi/2] ---------------
        AND  R4, R4, #0          ; R4 = bandera de signo (1 = negativo)
        ADD  R0, R0, #0
        BRzp SN_P1
        NOT  R0, R0
        ADD  R0, R0, #1          ; R0 = |x|  (x = -8.0 da x8000 = 32768)
        ADD  R4, R4, #1
SN_P1   LD   R1, SN_N2PI
        ADD  R2, R0, R1          ; |x| - 2pi  (siempre en [-25736, 7032])
        BRnz SN_P2               ; |x| <= 2pi: sin cambios
        ADD  R0, R2, #0          ; |x| -= 2pi          -> [0, 2pi]
SN_P2   LD   R1, SN_NPI
        ADD  R2, R0, R1          ; |x| - pi
        BRnz SN_P3               ; |x| <= pi: ya esta en [0, pi]
        ADD  R0, R2, #0          ; |x| -= pi           -> [0, pi]
        NOT  R4, R4              ; sen(x) = -sen(x-pi): invertir signo
        AND  R4, R4, #1
SN_P3   LD   R1, SN_NHALFPI
        ADD  R2, R0, R1          ; |x| - pi/2
        BRnz SN_P4
        NOT  R0, R0
        ADD  R0, R0, #1          ; -|x|
        LD   R1, SN_PI
        ADD  R0, R0, R1          ; pi - |x|            -> [0, pi/2]
SN_P4   ADD  R0, R0, R0
        ADD  R0, R0, R0          ; Q3.12 -> Q1.14 (<< 2)
        BRz  SN_EXIT             ; sen(0) = 0
        ST   R4, SN_SIGN
        ST   R0, SN_X
```

Código 2: Reducción del ángulo (Q3.12) a [0, π/2] y paso a Q1.14

Paso 3: núcleo racional. Aquí se evalúa la fórmula. En cada llamada a mulu o divf los operandos van en R0 y R1 y el resultado vuelve en R0. Primero se copia x a R1 y MULS15 calcula $u = x^2/2$, que se guarda en SN_U. Luego MULU_Q14 con A da $A \cdot u$; ese valor se niega (NOT y +1) y se le suma 1,0 (16384) para obtener $num = 1 - A \cdot u$, que se guarda en SN_NUM. Se recarga u, se multiplica por B y se suma 1,0 para obtener $den = 1 + B \cdot u$, que queda en R1. Se recupera num en R0 y DIVF calcula num/den. Por último, ese cociente pasa a R1, x vuelve a R0 y MULU_Q14 calcula $sen = x \cdot num/den$. Las últimas tres líneas saturan el resultado: se le resta 1,0 y, si la diferencia es positiva, se reemplaza por 1,0.

```
        ; ---- 2) Nucleo racional ------------------------------------
        ADD  R1, R0, #0
        JSR  MULS15             ; u = x*x/2   (redondeado)
        ST   R0, SN_U
        LD   R1, SN_A
        JSR  MULU_Q14           ; A*u
        NOT  R0, R0
        ADD  R0, R0, #1
        LD   R1, SN_ONE
        ADD  R0, R0, R1         ; num = 1 - A*u
        ST   R0, SN_NUM
        LD   R0, SN_U
        LD   R1, SN_B
        JSR  MULU_Q14           ; B*u
        LD   R1, SN_ONE
        ADD  R1, R0, R1         ; den = 1 + B*u
        LD   R0, SN_NUM
        JSR  DIVF               ; ratio = num/den  (<= 1.0)
        ADD  R1, R0, #0
        LD   R0, SN_X
        JSR  MULU_Q14           ; sen = x * ratio
        LD   R1, SN_NEG_ONE     ; saturar a 1.0
        ADD  R1, R0, R1
        BRnz SN_SGN
        LD   R0, SN_ONE
```

Código 3: Evaluación de x · (1 − A·u) / (1 + B·u)

Paso 4: signo y devolución de los registros. Si la bandera SN_SIGN vale 1, se niega R0 (NOT y +1) para devolver el seno con su signo. En SN_EXIT, que también es el destino de la salida temprana, se restauran R1 a R7 con LD. R7 se restaura al final para que el RET vuelva al programa principal. R0 es el único registro que conserva un valor nuevo: el seno.

```
        ; ---- 3) Signo ----------------------------------------------
SN_SGN  LD   R1, SN_SIGN
        BRz  SN_EXIT
        NOT  R0, R0
        ADD  R0, R0, #1

        ; ---- 4) Restaurar los registros del llamador ----------------
SN_EXIT LD   R1, SN_R1
        LD   R2, SN_R2
        LD   R3, SN_R3
        LD   R4, SN_R4
        LD   R5, SN_R5
        LD   R6, SN_R6
        LD   R7, SN_R7          ; direccion de retorno original
        RET
```

Código 4: Aplicación del signo, restauración de R1 a R7 y retorno

Debajo del código se reservan con .BLKW las posiciones de memoria para los registros guardados (SN_R1 a SN_R7) y para las variables temporales (SN_SIGN, SN_X, SN_U y SN_NUM), y con .FILL las constantes: 1,0 y −1,0 en Q1.14, −2π, π, −π y −π/2 en Q3.12, y los coeficientes A y B. Se guardan en memoria porque la LC-3 solo permite valores inmediatos de 5 bits en ADD y AND.

```
SN_R1       .BLKW #1
SN_R2       .BLKW #1
SN_R3       .BLKW #1
SN_R4       .BLKW #1
SN_R5       .BLKW #1
SN_R6       .BLKW #1
SN_R7       .BLKW #1
SN_SIGN     .BLKW #1
SN_X        .BLKW #1
SN_U        .BLKW #1
SN_NUM      .BLKW #1
SN_ONE      .FILL #16384          ; 1.0 (Q1.14)
SN_NEG_ONE  .FILL #-16384
SN_N2PI     .FILL #-25736         ; -2pi  (Q3.12)
SN_PI       .FILL #12868          ; pi    (Q3.12)
SN_NPI      .FILL #-12868
SN_NHALFPI  .FILL #-6434          ; -pi/2 (Q3.12)
SN_A        .FILL #3672           ; A = 0.2241
SN_B        .FILL #1814           ; B = 0.1107
```

Código 5: Variables temporales y constantes de SEN_Q12

Paso 5: mulu, preparación. MULS15 y MULU_Q14 son dos puertas de entrada a la misma rutina. Cada una guarda en MU_SQ el modo (1 para devolver hi, 0 para devolver 2·hi más el bit 14 de lo) y en MU_RND la constante de redondeo (2^14 o 2^13), y salta a MU_B. Allí se calcula a − b para dejar el menor de los dos números en R1 (multiplicador) y el mayor en R0 (multiplicando), intercambiándolos con R2 si hace falta. Si el multiplicador es 0, se salta a MU_ZERO y el producto es 0. Si no, se ponen en 0 los dos acumuladores, se desplaza el multiplicador una vez para que su bit 14 quede en el bit 15 y se carga R4 con 15, la cantidad de bits por revisar. El ciclo MU_SK desplaza el multiplicador hacia la izquierda, restando 1 al contador, hasta que aparece su primer bit en 1 (BRn). Los registros quedan con estos roles, y ni R6 ni R7 intervienen:

- R0: multiplicando (a, el mayor de los dos).
- R1: multiplicador (el menor); se examina su bit 15 en cada vuelta.
- R2 y R3: acumulador de 30 bits partido en dos palabras de 15 bits (lo y hi).
- R4: contador de bits que faltan; R5: temporal.

```
1  MULS15
2          AND  R2, R2, #0
3          ADD  R2, R2, #1
4          ST   R2, MU_SQ          ; modo: devolver hi
5          LD   R2, MU_H15
6          ST   R2, MU_RND         ; redondeo = 2^14
7          BR   MU_B
8  MULU_Q14
9          AND  R2, R2, #0
10         ST   R2, MU_SQ          ; modo: devolver 2*hi + bit14(lo)
11         LD   R2, MU_H14
12         ST   R2, MU_RND         ; redondeo = 2^13
13 MU_B    NOT  R2, R1             ; dejar el menor en R1
14         ADD  R2, R2, #1
15         ADD  R2, R0, R2         ; a - b
16         BRzp MU_OK
17         ADD  R2, R0, #0
18         ADD  R0, R1, #0
19         ADD  R1, R2, #0
20 MU_OK   ADD  R1, R1, #0
21         BRz  MU_ZERO            ; multiplicador 0 -> producto 0
22         AND  R2, R2, #0         ; acc_lo = 0
23         AND  R3, R3, #0         ; acc_hi = 0
24         ADD  R1, R1, R1         ; bit 14 del multiplicador -> bit 15
25         AND  R4, R4, #0
26         ADD  R4, R4, #15        ; 15 bits por revisar
27 MU_SK   ADD  R1, R1, #0         ; saltear los ceros iniciales
28         BRn  MU_LP
29         ADD  R1, R1, R1
30         ADD  R4, R4, #-1
31         BR   MU_SK
```

Código 6: Preparación de la multiplicación (mulu)

Paso 6: bucle de mulu y resultado. En cada vuelta se hace acc = 2·acc + (bit · a). Primero se duplica lo con ADD R2,R2,R2; si su bit 15 se enciende (BRzp no salta) hubo acarreo: se apaga ese bit con un AND contra 32767 y se hace hi = 2·hi + 1; si no, solo hi = 2·hi. Después se revisa el bit 15 del multiplicador: si es 1, se suma a a lo y, si de nuevo hay acarreo, se apaga el bit 15 y se suma 1 a hi. Se desplaza el multiplicador, se descuenta el contador y se repite mientras sea positivo (BRp). Al terminar, se suma la constante de redondeo con su posible acarreo y se arma el resultado: MULS15 devuelve hi; MULU_Q14 devuelve 2·hi más 1 si lo es 16384 o más. MU_ZERO devuelve 0.

```
MU_LP   ADD  R2, R2, R2          ; acc *= 2   (lo)
        BRzp MU_D1
        LD   R5, MU_M15
        AND  R2, R2, R5          ; acarreo: limpiar bit 15
        ADD  R3, R3, R3
        ADD  R3, R3, #1          ; hi = 2*hi + 1
        BR   MU_D2
MU_D1   ADD  R3, R3, R3          ; hi = 2*hi
MU_D2   ADD  R1, R1, #0          ; bit 15 del multiplicador
        BRzp MU_NB
        ADD  R2, R2, R0          ; acc_lo += a
        BRzp MU_NB
        LD   R5, MU_M15
        AND  R2, R2, R5          ; acarreo: limpiar bit 15
        ADD  R3, R3, #1          ; y subirlo a acc_hi
MU_NB   ADD  R1, R1, R1          ; multiplicador <<= 1
        ADD  R4, R4, #-1
        BRp  MU_LP
```

Código 7: Bucle principal de mulu (acc = 2·acc + bit·a)

```
        LD   R5, MU_RND          ; sumar el redondeo
        ADD  R2, R2, R5
        BRzp MU_RS
        LD   R5, MU_M15
        AND  R2, R2, R5
        ADD  R3, R3, #1
MU_RS   LD   R5, MU_SQ
        BRp  MU_HI
        ADD  R0, R3, R3          ; 2*hi
        LD   R5, MU_NEG1
        ADD  R5, R2, R5          ; lo - 16384
        BRn  MU_FIN
        ADD  R0, R0, #1          ; + bit 14 de lo
MU_FIN  RET
MU_HI   ADD  R0, R3, #0
        RET
MU_ZERO AND  R0, R0, #0
        RET
MU_SQ   .BLKW #1
MU_RND  .BLKW #1
MU_H14  .FILL #8192
MU_H15  .FILL #16384
MU_M15  .FILL #32767
MU_NEG1 .FILL #-16384
```

Código 8: Redondeo, resultado de mulu y constantes

Paso 7: divf. Es la división larga en binario. Se guarda −d en R3, el resto r = n en R2, el cociente q = 0 en R4 y un contador de 14 vueltas en R0. En cada vuelta se desplazan q y r a la izquierda, se calcula r − d en R1 y, si no es negativo (BRn no salta), ese valor pasa a ser el nuevo resto y se suma 1 a q. Pasadas las 14 vueltas se redondea: se duplica el resto y, si es mayor o igual que d, se suma 1 a q. El cociente se copia a R0 y la rutina vuelve con RET. Como mulu, divf solo usa R0 a R4 y no toca R6 ni R7.

```
1  DIVF
2          NOT  R3, R1
3          ADD  R3, R3, #1          ; R3 = -d
4          ADD  R2, R0, #0          ; r = n
5          AND  R4, R4, #0          ; q = 0
6          AND  R0, R0, #0
7          ADD  R0, R0, #14         ; contador
8  DV_LP   ADD  R4, R4, R4          ; q <<= 1
9          ADD  R2, R2, R2          ; r <<= 1
10         ADD  R1, R2, R3          ; r - d
11         BRn  DV_NS
12         ADD  R2, R1, #0
13         ADD  R4, R4, #1
14 DV_NS   ADD  R0, R0, #-1
15         BRp  DV_LP
16         ADD  R2, R2, R2          ; redondeo: 2r >= d ?
17         ADD  R1, R2, R3
18         BRn  DV_NR
19         ADD  R4, R4, #1
20 DV_NR   ADD  R0, R4, #0
21         RET
```

Código 9: División restauradora de 14 pasos (divf)

Verificación

Se compiló el programa en C y se comparó su salida con la función sin() de math.h en las 65 536 entradas posibles. El ensamblador se ejecutó con esas mismas 65 536 entradas en un simulador de LC-3 escrito para esta prueba y se comparó bit a bit con la salida del C. El error se midió contra el seno del ángulo exacto que representa cada entero de entrada, y la unidad de medida fue el LSB, es decir, 1/16384 ≈ 0,000061. Además, en cada ejecución se comprobó que R1 a R7 conservaran sus valores originales.

1.1.2. Resultados

1. División por cero y ángulo negativo

La división por cero no pudo ocurrir. La única división del programa es num/den y den = 1 + B·u nunca bajó de 1,0 (16384), porque B y u son positivos. Además, num ≤ den siempre, que es lo que divf necesita. Por eso divf no revisa el cero: la protección viene garantizada por la fórmula, no por una comprobación. Si se llamara a divf directamente con divisor 0, no se caería, pero devolvería un valor sin sentido.

Los ángulos negativos quedaron cubiertos: el programa calculó con el valor absoluto, recordó el signo en neg y lo invirtió al final, porque sen(−x) = −sen(x). El caso especial fue x = −8,0, cuyo valor absoluto no cabe en 16 bits con signo y se resolvió como se describió en la metodología.

2. Rango de ángulos aceptado

El programa aceptó cualquier entero de 16 bits con signo, de −32768 a 32767. En radianes eso es de −8,0 a casi +8,0 (7,99976), unos ±458°. El motivo es el formato: con 16 bits se reparten 1 bit de signo, 3 bits para la parte entera y 12 para los decimales. Con tres bits enteros se llega hasta 7,99976 pero no a 8,0, de modo que una vuelta completa (2π ≈ 6,28) sí cabe, pero 3π (9,42) no. Los ángulos mayores se tendrían que reducir antes de llamar a la función.

3. Formato de punto fijo

La entrada es Q3.12 con signo, de 16 bits (complemento a dos), y el valor real es el entero dividido entre 4096. La salida es Q1.14 con signo, y el valor real es el entero dividido entre 16384. La resolución de la entrada, es decir, el paso más pequeño que se puede representar, es 1/4096 ≈ 0,000244 rad. La resolución de la salida es 1/16384 ≈ 0,000061, unidad a la que se llama LSB.

4. Rango de la salida

La salida varió de −16384 a +16384, es decir, de −1,0 a +1,0, que es el rango natural del seno. Esto se comprobó en las 65 536 entradas. El programa nunca se pasa de ahí: la línea if (s > ONE) s = ONE; recorta el pequeño sobrepaso que podría producir la aproximación.

5. Precisión mínima y máxima

La precisión se refiere a qué tan cerca está el resultado del seno verdadero. Para medirla se comparó sen_q12(x) con el sin(x) real en todas las entradas y se calculó el error. La Tabla 2 resume los resultados y la Figura 1 muestra el error en todo el rango.

Tabla 2: Error de sen_q12 respecto de sen(x) en las 65 536 entradas

Medida	Valor
Mejor caso (error mínimo)	0 LSB. El resultado fue el entero más cercano al valor exacto en el 28,0 % de las entradas
Peor caso (error máximo)	3,75 LSB ≈ 0,00023 (0,023 %), en x = −32711 (−7,986 rad)
Error medio	1,21 LSB ≈ 0,00007
Error máximo por zona	3,75 LSB en [−8, −4), 3,71 LSB en [−4, 4) y 3,75 LSB en [4, 8)

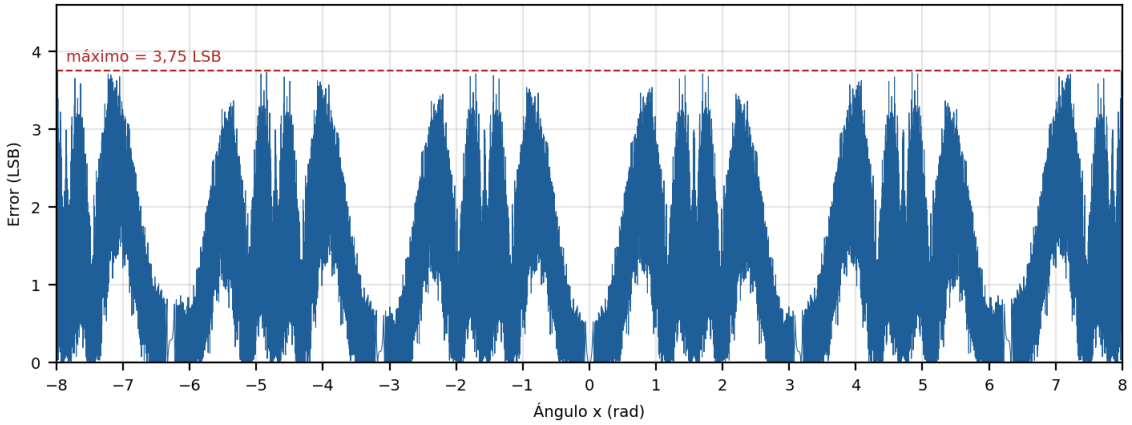


Figura 1: Error absoluto de sen_q12 respecto de sen(x), en LSB (1 LSB = 1/16384), para las 65 536 entradas

Un error de 0,00023 equivale a acertar unos 12 bits de los 14 posibles, o alrededor de 3,6 decimales correctos. Un LSB es la unidad mínima del formato, así que ni siquiera un cálculo perfecto bajaría de 0,5 LSB de error por el redondeo. La Tabla 3 compara algunos valores de control con el resultado exacto.

Tabla 3: Valores de control (resultado en Q1.14)

Ángulo	Entrada Q3.12	Resultado	Exacto
0	0	0	0
1,0 rad	4096	13785	13786,7
π/2	6434	16382	16384
π	12868	0	−0,1
5,0 rad	20480	−15713	−15711,0
−7,0 rad	−28672	−10762	−10764,1
+7,9998 rad	32767	16213	16210,2
−8,0 rad	−32768	−16211	−16209,6

6. Iteraciones

Los únicos bucles están en mulu y divf; la reducción del ángulo no tiene bucles. mulu da tantas vueltas como bits tiene el menor de los dos números y, como los números son de 15 bits, el máximo es 15 por llamada; se la llama 4 veces por cálculo. divf da siempre 14 vueltas, más un paso final de redondeo. La Tabla 4 muestra el total por cada cálculo de seno, medido sobre todas las entradas.

Tabla 4: Vueltas de bucle e instrucciones por cálculo de seno

	Mínimo	Máximo	Promedio
Vueltas dentro de mulu (las 4 llamadas)	6	52	≈ 48
Vueltas totales (con las 14 de divf)	20	66	≈ 62
Instrucciones de la LC-3	30	986	≈ 888

El mínimo de vueltas (entre las entradas que evalúan la fórmula) ocurrió con ángulos muy pequeños, como x = 1 (0,000244 rad), cuyas multiplicaciones tienen pocos bits. Las 5 entradas que se resuelven sin evaluar la fórmula (x = 0, ±π y ±2π) hacen 0 vueltas. El mínimo de 30 instrucciones corresponde a x = 0; las otras cuatro, x = +π, −π, +2π y −2π, necesitan 34, 37, 37 y 40 instrucciones, porque la salida ocurre directamente tras la reducción. Estos conteos incluyen las 14 instrucciones de guardado y restauración de R1 a R7 (7 ST y 7 LD) y no incluyen las 4 instrucciones del programa de prueba (LD, JSR, ST y HALT).

7. Pruebas con ángulos ingresados por el usuario

Además del análisis de las 65 536 entradas, el programa en C permite que el usuario ingrese un ángulo en radianes. El programa lo convierte a Q3.12, calcula el seno en Q1.14 y muestra el ángulo ya convertido, el resultado como entero y como valor real, el seno real que entrega la función sin() para comparar y el número de iteraciones de los bucles de mulu y divf. Se probaron tres ángulos que ilustran situaciones distintas (Figuras 2 a 4). Las últimas líneas de cada captura corresponden al análisis general de las 65 536 entradas (Tabla 2) y no dependen del ángulo ingresado.

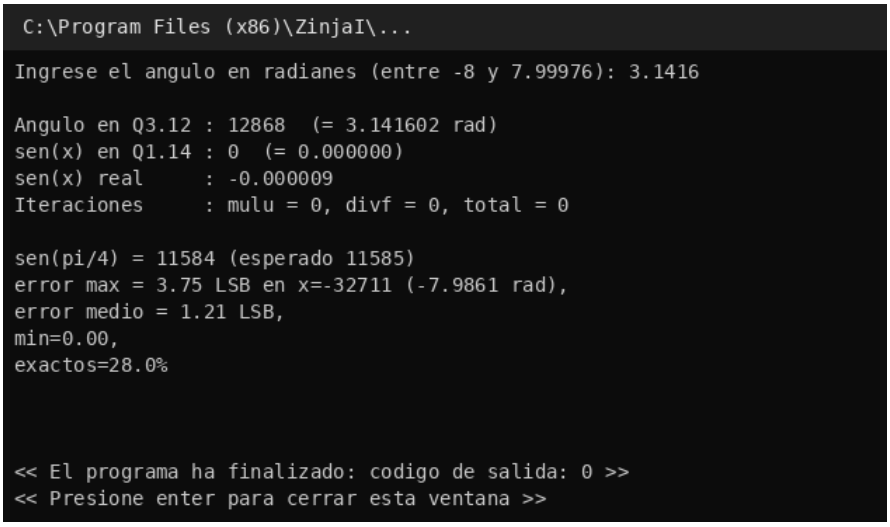


Figura 2: Prueba con x = 3,1416 rad (≈ π)

Caso 1: x = 3,1416 rad. El ángulo 3,1416 se convierte a 12868 en Q3.12, que es exactamente la constante π del programa (3,141602 rad). En la reducción, ese valor no supera 2π ni π, pero sí π/2, así que se calcula π − x = 0. Con el ángulo reducido en 0, el programa devuelve 0 de inmediato, sin llamar a mulu ni a divf, y por eso los tres contadores de iteraciones valen 0. El resultado es correcto: el seno real de 3,141602 es −0,000009, equivalente a −0,15 unidades de las 16384 de la salida, que al redondearse a entero da 0.

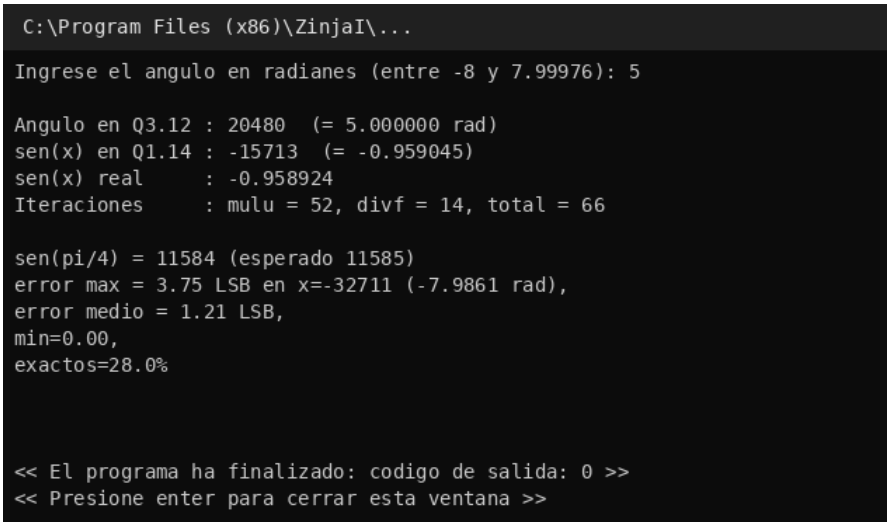


Figura 3: Prueba con x = 5 rad

Caso 2: x = 5 rad. El ángulo es 20480 en Q3.12. Como 5 es mayor que π, se le resta π: 20480 − 12868 = 7612 (≈ 1,858 rad) y se invierte el signo guardado, porque sen(x) = −sen(x − π). Como 7612 supera π/2 (6434), se refleja con sen(x) = sen(π − x): 12868 − 7612 = 5256 (≈ 1,283 rad), que pasa a 21024 en Q1.14. El resultado es −15713 (−0,959045) contra −0,958924 del seno real, es decir, una diferencia de unas 2,0 unidades de salida (el valor exacto es −15711,0). Las iteraciones fueron 52 en mulu y 14 en divf, 66 en total, que es el máximo de la Tabla 4, porque con 1,283 rad las multiplicaciones usan operandos de muchos bits.

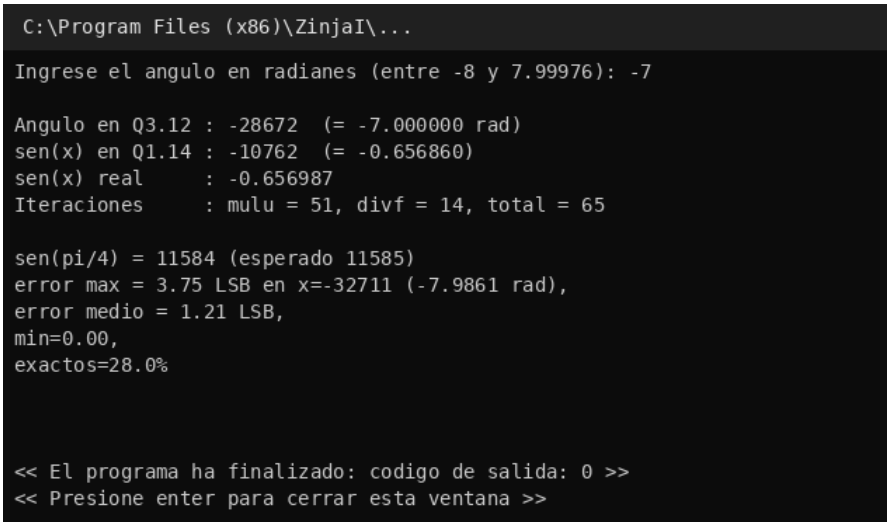


Figura 4: Prueba con x = −7 rad

Caso 3: x = −7 rad. El ángulo es −28672 en Q3.12. Al ser negativo, el programa trabaja con su valor absoluto (28672, es decir, 7,0 rad), que supera 2π, y le resta una vuelta completa: 28672 − 25736 = 2936 (≈ 0,717 rad). Ese ángulo ya está entre 0 y π/2 y no necesita más reducción, así que calcula sen(0,717) y al final invierte el signo, porque sen(−x) = −sen(x). El resultado es −10762 (−0,656860) contra −0,656987 del seno real, una diferencia de 2,1 unidades de salida (el valor exacto es −10764,1), por debajo del error máximo de 3,75. Las iteraciones fueron 51 en mulu y 14 en divf, 65 en total.

En conjunto, los tres casos cubren una salida anticipada por ángulo reducido igual a 0 (caso 1), una resta de π con inversión de signo y reflexión π − x (caso 2) y un ángulo negativo con resta de una vuelta completa 2π (caso 3). En todos, el resultado de la función coincide con el seno real dentro del error esperado y el número de iteraciones queda dentro del rango medido en la Tabla 4.

Resumen de cifras

Tabla 5: Resumen de cifras del programa

Dato	Valor
Formato de entrada	Q3.12 (entero × 4096)
Formato de salida	Q1.14 (entero × 16384)
Rango de ángulos	−8 a casi 8 radianes
Error máximo	≈ 3,75 de 16384 unidades
Error medio	≈ 1,21 unidades
Tamaño del ensamblador	187 palabras de memoria
Instrucciones por cálculo	≈ 888 en promedio

1.1.3. Discusión

Los resultados muestran que ampliar el rango de entrada a ±8 radianes, más de una vuelta completa, no degrada la precisión de la salida. El error máximo (3,75 LSB) proviene del núcleo racional, y las reducciones por 2π y por π solo agregan el error de redondear las constantes, que es despreciable. El error además es prácticamente igual en todas las zonas del rango (Tabla 2).

El costo del rango mayor es la resolución de la entrada: 1/4096 rad. Como el seno tiene pendiente de hasta 1, dos ángulos de entrada consecutivos pueden dar salidas separadas hasta por 4 LSB (1/4096 = 4/16384). Este error de cuantización del ángulo no figura en la Tabla 2, que mide contra el seno del ángulo exacto que representa cada entero. Si se parte de un ángulo real y se lo redondea a Q3.12 antes de llamar a la función, ese redondeo puede sumar hasta 2 LSB más a la salida.

La subrutina ocupa 187 palabras de memoria (181 de la subrutina con sus datos y 6 del programa de prueba) y ejecuta unas 888 instrucciones en promedio por cálculo. El guardado y la restauración de R1 a R7 agregan solo 14 instrucciones por llamada, un costo pequeño frente al total y necesario para no dañar al programa principal. Como R6 y R7 no pueden usarse, mulu se reescribió multiplicando de izquierda a derecha, con el multiplicando en una sola palabra y el acumulador en dos. El ensamblador coincide con el C en todas las entradas, lo que confirma que la traducción conserva el resultado bit por bit.

Las cifras de instrucciones y de tamaño se midieron en un simulador de LC-3 propio, por lo que conviene repetir la medición en el simulador de la cátedra antes de citarlas

.

2. CONCLUSIONES

Se calculó el seno de un ángulo en C y en ensamblador de la LC-3, con entrada en formato Q3.12 (±8 radianes) y salida en formato Q1.14, usando solo sumas, restas y desplazamientos de bits.

La aproximación racional alcanzó un error máximo de 3,75 LSB (≈ 0,023 %) y un error medio de 1,21 LSB sobre las 65 536 entradas posibles, y la versión en ensamblador coincidió bit a bit con la de C.

La entrada Q3.12 cubre más de una vuelta completa (±8 radianes) con una resolución de 1/4096 rad, lo que se logró agregando al programa la reducción por 2π. La subrutina respeta la convención de llamada: recibe y devuelve el dato en R0, no usa R6 ni R7 como registros de trabajo y restaura R1 a R7 antes de volver.

En resumen, el programa calcula el seno aproximándolo con una fracción de polinomios, usando números enteros en lugar de decimales, y construye a mano la multiplicación y la división porque la computadora no las tiene.

3. REFERENCIAS BIBLIOGRÁFICAS

[1] S. Harris and D. Harris, Digital design and computer architecture: arm edition: Morgan Kaufmann, 2015.

[2] Y. N. Patt and S. J. Patel, Introduction to Computing Systems: From Bits & Gates to C & Beyond, 2nd ed.: McGraw-Hill, 2004.

[3] J.-M. Muller, Elementary Functions: Algorithms and Implementation, 3rd ed.: Birkhäuser, 2016.

[4] J. F. Hart et al., Computer Approximations: John Wiley & Sons, 1968.

[5] L. N. Trefethen, Approximation Theory and Approximation Practice: SIAM, 2013.